TDR-W: A Weighted Framework for Prioritizing Technical Debt Based on Development Pain and Static Costs

Mihai-Constantin Ban

***Abstract*—This report presents a new method, called the TDR-W framework, for prioritizing technical debt (TD). It is designed to work better than old methods, which often get prioritization wrong (most do not have much practical impact, focusing more on the theoretical aspect, but there are other issues as well). The core idea is simple: debt risk is a combination of how much effort it takes to fix (the principal) and how much pain it is currently causing (the interest). This idea is defined with a weighted formula:** $TDR\_Score = (Effort) \times (w_{commits} \cdot Recent\_Commits + w_{bugs} \cdot Recent\_Bugs)$**. This report provides a methodology for gathering these numbers by mining a project's coding history. It explains how to use the Lizard tool to estimate the 'Effort', the PyDriller library to count 'Recent Commits', and a practical workaround (called Heuristic-Based Fix-Mapping) to find 'Recent Bugs' from commit messages. A case study on several major open-source projects demonstrates how this formula helps find the real problem areas (hotspots) and helps to safely ignore benign debt, which is messy code that is not actually hurting anyone.**

***Index Terms*—Technical Debt, Software Metrics, Risk Management, Software Reliability, Prioritization.**

I. Introduction

THE primary motivation for a new framework is the inadequacy of current tools and metrics in predicting real problems. Many studies have tried to find a link between the messiness of code (measured by tools like SonarQube) and real-world developer problems (like how long it takes to deliver a feature). The results are consistently weak or inconsistent. When researchers checked if messy code led to slower delivery times, they got mixed results. In some cases, it did, in others, it had no meaningful impact, and in two cases, it actually had a negative impact.

Their conclusion was clear: the relationship between code messiness (TD) and delivery time (lead time) is either very weak or nonexistent. This shows a big flaw in how debt is typically perceived. It proves that the cost of fixing debt (the principal) is totally separate from the pain it causes (the interest). If they were linked, the most expensive items could just be fixed first, but they are not. A piece of code can be very expensive to fix but cause zero pain because no one ever touches it. This is why the TDR-W framework is built on the idea that "cost" and "pain" are two different things.

M.-C. Ban is with the Department of Computer Science, Babeș-Bolyai University, Cluj-Napoca, Romania.

II. Related Work

A. The problem with prioritizing

Beyond bad metrics, the whole field of "how to prioritize" is a challenge. Researchers who performed a systematic literature review on this topic found that no one can agree on the best way to do it [4]. First, it is too complex. One review found 53 unique, individual factors that different researchers use to prioritize debt. This makes it impossible for a real team to just pick a model and use it.

Second, most methods fail to answer the real question. Most methods only help to decide "which piece of debt should be fixed first?" But the real question teams face is, "should this debt be fixed or should this new feature be built?". Third, most methods ignore real-world limits. Another review found that only 16.67% of methods even consider a team's limited resources, like time and developer availability.

This research shows a new framework is needed that is simple (not based on 53 factors), contextual (weighs debt against new features), and measures both static "cost" and real-world "pain".

B. Defining development pain

"Pain" is not just a feeling, it is something that can be measured by looking at a project's history. The TDR-W formula breaks "pain" into two measurable parts backed by research:

1) Pain as unreliability (Recent bugs): The best reason for using bug history comes from a 2016 study by MacCormack & Sturtevant [13]. They suggested that messy, tangled code (high coupling) in the system's core was directly responsible for a huge number of bug reports. This gives a strong, citable reason to use Recent Bugs as a key part of the "Pain Score". It is a direct measure of where debt is causing the product to fail.

2) Pain as friction (Recent commits): Technical debt also creates a constant tax on development. A 2018 study by Besker et al. [8] measured this and found that developers waste, on average, "23% of their total development time" just working around technical debt. This time is wasted when developers are actively working in that part of the code, trying to understand it, writing extra tests, and avoiding issues. The Recent Commits metric is a direct way to measure this friction. Debt in a file that hasn't been touched in years isn't wasting 23% of anyone's time, but debt in a core file that is changed every day is a hotspot. Another research proposal by Gupta

(2018) [2] also supported this idea of linking TD to changesets.

III. The TDR-W Model

A. Conceptual framework: Principal and interest

The TDR-W model is based on a simple analogy:

- **The principal (Effort to fix):** This is the one-time cost, in hours or days, to completely fix the debt and remove it from the code. This is a standard concept in TD management [7].
- **The interest rate (Pain score):** This measures the ongoing cost of doing nothing. It is the "interest" being paid every single day in the form of wasted time, developer frustration, and customer-facing bugs.

A high Pain Score means the code is an active issue, it is breaking things or blocking work right now. A low Pain Score means the code is messy but harmless, it is ugly, but it is not causing any immediate problems. The TDR score is calculated by multiplying the effort and the pain score: $TDR = Effort \times Pain$. This multiplication automatically finds the real risk and handles cases static-only tools get wrong:

1) **High-effort, low-pain debt:** A complex, old module that no one ever touches (ex: Effort=5000, Pain=1) results in a low score.
2) **Low-effort, high-pain debt:** A simple, badly designed function that is changed daily (ex: Effort=50, Pain=100) results in a high score.

This acts as a pain filter, letting teams focus limited time only on debt that is actively hurting them.

B. The TDR-W formula

The formal TDR-W formula is:

$$TDR = E_{fix} \times (w_c \cdot C_{recent} + w_b \cdot B_{recent}) \quad (1)$$

Where:

- E_{fix} is the Effort to fix (Principal).
- C_{recent} is Recent Commits (Friction proxy).
- B_{recent} is Recent Bugs (Unreliability proxy).
- w_c and w_b are weights for friction and unreliability.

These weights allow a company to apply strategy to the model. For example, an **Aggressive Startup** might set $w_c = 1.0, w_b = 1.0$, treating friction as equal to instability. A **Mature Enterprise** might set $w_c = 0.5, w_b = 2.0$, implying a customer-facing bug is 4x more harmful than developer friction.

IV. MSR-Based Methodology (Mining Software Repositories)

This chapter provides the methodology for gathering the three numbers in the formula using Python scripts with Lizard and PyDriller.

A. Instrumentation 1: Effort proxy (Lizard)

The formula needs an Effort to fix score. Tools like SonarQube provide "rough estimates," but the Lizard tool only provides complexity metrics like NLOC and CCN. We propose the **Lizard Effort Proxy (LEP)** model. Fixing code involves the volume of code to be read and its complexity. The LEP formula is:

$$LEP = \sum NLOC + (k \times \sum CCN) \quad (2)$$

Where k is a complexity penalty (ex: $k = 5$) reflecting that complex code is non-linearly harder to fix. The script runs Lizard, aggregates NLOC and CCN per module, and applies the formula.

B. Instrumentation 2: Recent commits (PyDriller)

The Recent Commits score is a simple count of how many times each module has been changed in a "recent" period (ex: the last 6 months).

C. Instrumentation 3: Recent bugs (HBFM)

The standard academic way to link bugs to code (SZZ algorithm) is complicated, slow, and unmaintained. We propose **Heuristic-Based Fix-Mapping (HBFM)**. The TDR-W formula just needs to find hotspots of unreliability, it is not necessary to know when bugs were introduced, only where they are being fixed. HBFM identifies a commit that looks like a bug fix (via message heuristics) and counts the files in it. This gives a practical measure of reliability strain at a fraction of the cost.

V. Case Study: Real-World Data Analysis

Data was gathered from seven open-source repositories: `apache/commons-cli`, `apache/commons-lang`, `psf/requests`, `zxing/zxing`, `pallets/flask`, `facebook/react`, and `chartjs/Chart.js`. The analysis used weights $w_c = 0.5$ and $w_b = 1.0$.

A. Results: TDR hotspot analysis

Table I shows the top 10 highest-risk modules from the combined dataset.

B. Findings

Finding 1: Effort alone is misleading. In `commons-lang`, traditional tools would prioritize the `builder` module (Effort: 7764) over `function` (Effort: 1501). However, TDR analysis shows `function` (TDR: 45,780) is a major pain source (30.5 Pain Score), while `builder` is stable (8.0 Pain Score).

Finding 2: Finding real problem areas. The `react-server` module (Rank 2) highlights the model's value. Despite having half the Effort of Rank 1, its Pain Score of 140.0 (driven by 58 recent bugs) makes it a critical priority. This is something static-only tools would miss.

Finding 3: Ignoring benign debt. Table II shows modules with high effort but negligible pain. For example, `zxing/oned` has a huge Effort score (7,187) but a Pain Score of only 0.5. TDR-W calculates a low risk score (3,593), identifying it as benign debt that can be safely ignored.

TABLE I
TOP 10 TDR-W HOTSPOTS ACROSS ALL ANALYZED PROJECTS

Project	Module	TDR Score	Effort	Pain	Commits	Bugs
React	react-reconciler	6,024,431	59,354	101.5	171	16
React	react-server	3,169,460	22,639	140.0	164	58
Lang	commons-lang3	2,767,800	31,632	87.5	145	15
React	babel-plugin/HIR	844,984	10,696	79.0	136	11
React	react-client	762,454	9,902	77.0	84	35
React	fiber	645,028	13,724	47.0	80	7
React	devtools/views	461,155	6,190	74.5	133	8
React	react-dom-bindings	341,082	18,949	18.0	32	2
CLI	commons-cli	315,849	6,133	51.5	89	7
React	babel-plugin/Val	293,002	5,581	52.5	101	2

TABLE II
EXAMPLES OF BENIGN DEBT (HIGH-EFFORT, LOW-PAIN)

Project	Module	Effort	Pain	TDR
zxing	core/oned	7,187	0.5	3,593
Chart.js	plugin.filler	1,282	0.5	641
Lang	lang3/exception	1,188	0.5	594

VI. DISCUSSION AND FUTURE WORK

A. Implications for practitioners

TDR-W serves as the quantitative engine for a "Prioritization Matrix". By plotting TDR_Score (Risk) against Business_Value, teams can solve the "debt vs. features" dilemma. Items with High TDR and High Business Value become immediate priorities.

B. Limitations

- **Effort Guess:** The LEP model is an unvalidated heuristic. The k-factor is a proposal and Lizard does not provide remediation effort.
- **Bug Guess:** HBFM relies on commit messages. It is prone to false negatives (vague messages) and false positives (fixing typos).
- **Subjectivity:** Weights (w_c, w_b) are strategic choices rather than objective constants.

C. Future work

Future work should focus on:

1) **Validating the effort formula:** Compare LEP scores against real developer effort (in hours) for refactoring tasks to tune the k-factor.
2) **Improving the bug-finding script:** Use issue-tracker APIs (Jira/GitHub) to confirm issue types (ex: ensure issue "AIRFLOW-123" is Type=Bug), eliminating false positives.
3) **A/B testing:** Run a longitudinal experiment where Team A uses traditional prioritization and Team B uses TDR-W. If Team B shows shorter lead times and fewer bugs, the framework is proven.

REFERENCES

[1] D. Saraiva et al., "Technical Debt Tools: A Systematic Mapping Study," in *Proc. 23rd ICEIS*, vol. 2, pp. 88-98, 2021.
[2] K. Gupta, "Measuring the Impact of Technical Debt on Development Effort in Software Projects," Research Paper, 2018.
[3] B. Paudel et al., "Towards Measuring the Impact of Technical Debt on Lead Time," arXiv:2406.01578, 2024.
[4] R. Alfayez et al., "A Systematic Literature Review of Technical Debt Prioritization," in *TechDebt '20*, pp. 1-10, ACM, 2020.
[5] M. Wiese et al., "Establishing Technical Debt Management," arXiv:2508.15570, 2025.
[6] N. Brown et al., "Managing technical debt in software-reliant systems," *FSE/SDP Workshop*, pp. 47-52, 2010.
[7] C. Seaman and Y. Guo, "Measuring and monitoring technical debt," *3rd Int. Workshop on Managing Tech. Debt*, pp. 1-7, 2011.
[8] T. Besker et al., "Technical debt cripples software developer productivity," *Proc. 2018 Int. Conf. on Tech. Debt*, pp. 105-114, 2018.
[9] V. Lenarduzzi et al., "A systematic literature review on Technical Debt prioritization," *J. Syst. Softw.*, vol. 171, p. 110827, 2021.
[10] D. Sculley et al., "Hidden technical debt in machine learning systems," in *NeurIPS*, pp. 2503-2511, 2015.
[11] J. D. Morgenthaler et al., "Searching for build debt," in *3rd MTD*, pp. 1-6, IEEE, 2012.
[12] T. Besker et al., "Technical debt cripples software developer productivity," in *TechDebt '18*, pp. 105-114, 2018.
[13] A. MacCormack and D. J. Sturtevant, "Technical debt and system architecture," *J. Syst. Softw.*, vol. 120, pp. 170-182, 2016.
[14] T. Besker et al., "The influence of Technical Debt on software developer morale," *J. Syst. Softw.*, vol. 167, p. 110586, 2020.
[15] Z. Li et al., "A systematic mapping study on technical debt and its management," *J. Syst. Softw.*, vol. 101, pp. 193-220, 2015.